

Instructions pour le TP0 – Simulateur SIMSE

L'objectif de ces simulations est de découvrir l'impact de vos décisions concernant l'approche différente de gestion de projet nécessaire selon le cycle de vie logiciel que votre employeur pourrait utiliser. Conséquemment vous allez comprendre que pour le gestionnaire de projet logiciel, le choix d'un cycle de vie a un grand impact sur sa manière de gérer un projet logiciel. Donc, prenez le temps de lire et de vous documenter (soit à l'aide des références suivantes, soit en fouillant sur le Web), sur le fonctionnement particulier de chaque cycle de vie. Ainsi, en vous documentant sur leur fonctionnement vous allez mieux comprendre comment avoir du succès dans le jeu de simulation pour chaque cas de figure.

Il existe des vidéos en ligne (en anglais) qui sont des tutoriels, et il est recommandé de les regarder avant d'exécuter les simulations. Voir les 3 liens au bas de cette page (#1=11m31s; #2=17m37s; #3=8m26s, total de 38m34s) :

<https://ics.uci.edu/~emilyo/SimSE/downloads.html#Videos>

Dans cette même page, il y a des manuels (3) d'instructions dans la section « Docs »

1.1 Références pour le cycle de vie en Cascade (en anglais = « Waterfall ») :

1. Boehm, B.W., *Software Engineering Economics*. 1981, Upper Saddle River, NJ: Prentice Hall, Inc.
2. Brooks, F.P., *The Mythical Man-Month: Essays on Software Engineering*. 2 ed. 1995, Boston, MA: Addison-Wesley. 336.
3. Bryan, G.E., *Not All Programmers are Created Equal*, in *Software Engineering Project Management*, R.H. Thayer, Editor. 1997, IEEE Computer Society: Los Alamitos, CA. p. 346-355.
4. Curtis, B., H. Krasner, and N. Iscoe, *A Field Study of the Software Design Process for Large Systems*. Communications of the ACM, 1998. **31**(11): p. 1268-1287.
5. Sackman, H., W.J. Erikson, and E.E. Grant, *Exploratory Experimental Studies Comparing Online and Offline Programming Performance*. Communications of the ACM, 1968. **11**(1): p. 3-11.
6. Tvedt, J.D., *An Extensible Model for Evaluating the Impact of Process Improvements on Software Development Cycle Time*. 1996, Ph.D. Dissertation, Arizona State University.

1.2 Références pour le cycle de vie incrémental :

1. Boehm, B.W., *A Spiral Model of Software Development and Enhancement*. IEEE Computer, 1988. **21**(5): p. 61-72.
2. Gilb, T., *Evolutionary Delivery versus the Waterfall Model*. ACM SIGSOFT Software Engineering Notes, 1985: p. 49-61.
3. Laman, C. and V. Basili, *Iterative and Incremental Development: A Brief History*. IEEE Computer, 2003. **36**(6): p. 47-56.
4. Royce, W., *TRW's Ada Process Model for Incremental Development of Large Software Systems*, in *Proceedings of the 12th International Conference on Software Engineering*. 1990. p. 2-11.
5. Scacchi, W., *Process Models in Software Engineering*, in *Encyclopedia of Software Engineering*, J. Marciniak, Editor. 2001, Wiley.

1.3 Références pour le cycle de vie de prototypage rapide :

1. Gordon, V.S., Bieman, J.M. *A Rapid prototyping : Lessons Learned*. IEEE Software, 1995. **12**(1): p. 85-95.

2. Kordon, F., Luqi. *An introduction to rapid system prototyping*. IEEE Transactions on Software Engineering, 2002. **28**(9): p. 817-821.
3. Gomaa, H. and Yusuf, Y.Y., *The impact of Rapid prototyping on user requirements*. ACM SIGSOFT, 1983.
4. Hartson, H.R. Smith, E.C. *Rapid prototyping in human computer interface development*, Interacting with Computers, 1991. **3**(1): p. 51-91.

1.4 Références pour le cycle de vie RUP (acronyme = Rational Unified Process) :

1. Kruchten, P. *The Rational Unified Process: An Introduction* (2nd Edition). 2000: Addison-Wesley.
2. Kruchten, P. *Going over the waterfall with the RUP*. Rational Edge Magazine, 2001, 16 p.
3. Ambler, S.W. *A manager's Introduction to Rational Unified process (RUP)*. AmbySoft, 2005, 18 p.
4. Smith, J. *A comparison of RUP and XP*. Rational Software White Paper, 2002, 22 p.
5. Parnas, D., Clements, P.C. *A rational Design process*, IEEE Transactions, SE-12(2), February 1986.

1.5 Références pour le cycle de vie Extreme Programming (XP) :

1. Cockburn, A. and L. Williams, *The Costs and Benefits of Pair Programming*, in *Extreme Programming Examined*. 2001, Addison Wesley.
2. Wake, W.C., *Extreme Programming Explored*. 2002, Boston, MA: Addison-Wesley.
3. Juric, R. *Extreme Programming and its development practices*, Proceedings of the 22nd International Conference on Information Technology Interfaces, 13-16 Juin 2000, p. 97-104.
4. Ji, F. and Sedano, T. *Comparing extreme programming and waterfall project results*, Proceedings of the 24th IEEE-CS Conference on Software Engineering Education and Training (CSEE&T), 22-24 Mai 2011, p. 482-486.
5. Beck, K. *Embracing change with extreme programming*, Computer, 32(10), Oct 1999: p. 70-77.

2- Installez SIMSE sur votre poste de travail :

2.1 MacOS :

1. Vérifiez que vous avez le Java SE Development kit 17: Java jdk-17_osx-x64.bin d'installé pour votre macOS. Sur mac la dernière version de Java fonctionne bien avec le jeu;
2. Obtenez les fichiers (.jar) de chacun des 5 cycles de vie et du module d'inspection dans la section **TPO-SIMSE de Moodle** et mettez-les sur votre bureau ;
3. Démarrez un terminal :
 1. Changez de répertoire avec la commande `cd desktop` ;
 2. Démarrez un jeu avec la commande `java - jar waterfall.jar` ;
 3. Changez **waterfall** par le nom des autres cycles de vie pour les démarrer.

2.2 Windows 10 :

1. Les dernières versions de Java ne fonctionnent pas bien avec ce jeu. Vérifiez que vous avez le Java SE 17 (LTS) d'installé sur votre équipement. Si vous avez une version plus récente de Java qui ne fonctionne pas, enlevez-la et remplacez-la par la version 17 (vous pourrez la remettre après le cours);
2. Obtenez les fichiers (.jar) de chacun des 5 cycles de vie et du module d'inspection dans la section **TPO-SIMSE de Moodle** et mettez-les sur votre bureau ;
3. Double-cliquez sur le fichier .jar et le jeu débute.

3- Faites votre formation au jeu SIMSE en regardant les vidéos que nos étudiants ont rendu disponible sur Youtube (merci à Youssef) :

- Débutez avec le tutoriel du simulateur **SIMSE** :
<https://www.youtube.com/watch?v=IJpHPJrj6Pc&t=13s>
- Formez-vous sur l'outil de branchement SIMSE pour répéter une partie de la simulation à partir d'un point précis et prendre une autre décision : <https://www.youtube.com/watch?v=t-g2huTfzsQ&t=5s>
- Étudiez l'outil explicatif SIMSE qui permet de révéler des détails de la simulation :
<https://www.youtube.com/watch?v=iiDJmBs-6UU&t=3s>

4- Conservez des notes pour le Quiz en ligne :

Pour ce TP0, vous devez effectuer la simulation de tous les cycles de vie et prendre des notes personnelles qui vous seront utiles pour répondre aux questions du Quiz:

- Notez votre meilleure note obtenue et le nombre de 'ticks' (c.-à-d. jours) et notez le nombre maximum de 'ticks' à ne pas dépasser pour obtenir le meilleur score;
- Pour tous les cycles de vie, notez la séquence des étapes (stratégie) à suivre pour obtenir le meilleur résultat;
- Pour tous les cycles de vie, retenez le nom du système à développer;
- Notez l'impact de l'assignation des personnes aux tâches selon leurs compétences/expérience et d'impliquer les bonnes personnes au bon moment;
- Notez l'importance des revues pour découvrir les défauts qui se cachent dans le logiciel (tentez de voir, avec l'outil explicatif combien de défauts se cachent dans votre projet);
- Notez l'impact d'acheter ou pas des outils d'ingénierie;
- Analysez les objets (par exemple: l'énergie, le 'Mood' et le 'Payrate' des employés, les caractéristiques des livrables (Nb d'erreurs détectées, Nb d'erreurs non détectées, %d'erreur, %Complété) et l'impact de l'utilisation d'outils de génie logiciel (Coût, facteur d'augmentation de la productivité et Facteur de réduction d'erreurs);
- Dans le cycle de vie **waterfall** :
 - a) Quel est l'effet de congédier André dès le début du projet?
 - b) Qu'est-ce qu'un débutant comme Roger peut et ne peut pas faire comme tâche?
 - c) Qu'arrive-t-il si vous ne complétez pas complètement une étape et démarrez quand même l'étape suivante (ex. phase des exigences et/ou phase de conception)?

- d) Qu'arrive-t-il (2 impacts) quand on met trop d'employés au travail sur une même tâche?
 - e) À quel moment il est préférable d'effectuer la revue d'un artéfact?
 - f) Quel est l'impact de ne pas préparer les tests avant de les effectuer?
 - g) Comment doit-on traiter une demande de changement efficacement dans ce cycle de vie?
 - h) Quel est l'effet de donner une prime à un employé?
 - i) Quel est l'effet d'accorder une augmentation de salaire à un employé?
 - j) Est-il intéressant d'acheter des outils?
- Dans le cycle **RUP** :
- a) Quelle approche de prototypage est récompensée dans ce jeu ? Combien (environ) de cas d'utilisation ont été intégrés dans chacun lorsque vous les avez soumis et que le client en était satisfait?
 - b) Combien de fois pouvez-vous acheter des outils?
 - c) Quels sont les artéfacts produits lors de l'«Inception» (inception=démarrage)?
 - d) Quelles sont les activités principales des phases de démarrage, de l'élaboration et de développement?
 - e) Quel est l'effet de faire trop ou pas assez d'itérations dans une phase?
 - f) Quelle phase prend le plus de 'ticks' d'horloge (i.e. nécessite plus de temps et d'efforts)?
 - g) Quels ont été les principaux compromis impliqués dans l'affectation du personnel dans ce jeu?
- Dans le cycle de vie **Extreme Programming (XP)** :
- a) Quelles sont les activités d'une réunion de planification?
 - b) Que se passe-t-il si vous n'impliquez pas le client dans la réunion de planification de la version?
 - c) Que se passe-t-il si vous excluez un développeur dans une réunion de planification d'itération? Quel est l'effet sur la participation de ce développeur au reste de l'itération? Pourquoi pensez-vous qu'il en est ainsi?
 - d) Quelles sont les deux principales activités réalisées lors d'une réunion de planification d'itération?
 - e) Que se passe-t-il si vous ne faites pas les tests unitaires au bon moment?
 - f) Quelle est la différence entre effectuer des tests unitaires après chaque activité de développement de code (programmation, refactorisation et intégration) et les effectuer uniquement après l'intégration ? Pourquoi?
 - g) À quel moment il faut créer des tests d'acceptation? Quel est l'effet de ne pas les créer à ce moment idéal, et pourquoi cet effet se produit-il?
 - h) Que se passe-t-il si vous n'incluez pas le client dans la création des tests d'acceptation?
 - i) Quelle est la séquence idéale d'étapes à suivre dans chaque itération?
 - j) Est-ce que le budget est important dans ce jeu?

- Dans le cycle de vie **incrémental** :
 - a) Quel attribut d'artefact semble être le plus important et avoir le plus d'impact sur le résultat du jeu (par exemple, inflexibilité, difficulté, variabilité, etc.) ? Pourquoi ?
 - b) Quel est l'effet de sauter une ou plusieurs phases de la documentation (exigences/conception) sur un ou plusieurs modules?
 - c) Comment la soumission anticipée d'un projet partiellement terminé affecte-t-elle votre travail sur le reste du projet ?
 - d) Quel attribut s'améliore si on effectue les exigences avant la conception? Essayez de sauter l'étape des exigences pour voir l'effet.

- Dans le cycle de vie **prototypage rapide** :
 - a) Quelle est, selon vous, la manière la plus efficace d'aborder le prototypage dans le jeu (quelle quantité de prototypage est la meilleure, à quel moment du processus doit-il être effectué, etc.)?
 - b) Quel est l'un des avantages les plus importants de laisser le client évaluer le prototype?
 - c) Quel est l'effet de faire trop peu ou beaucoup trop de prototypage sur le projet?
 - d) Pour quels types de projets logiciels une approche de prototypage rapide est-elle la plus adaptée et pourquoi?
 - e) Quels sont les effets de l'omission totale du prototypage sur le document d'exigences qui en résulte? En d'autres termes, quelle est la différence entre un document d'exigences pour un système qui a été prototypé et un autre qui ne l'a pas été?
 - f) Quel est l'effet d'un prototypage trop limité sur la conception et le code qui sont développés ultérieurement?
 - g) Essayez tous les langages et identifiez le bon langage pour prototyper versus l'implémentation finale. Et aussi regardez l'effet de l'utilisation du même langage pour ces deux activités;

- Dans le module **d'Inspection** :
 - a) Quelle semble être la taille idéale d'une équipe d'inspection?
 - b) Combien de temps une inspection devrait-elle durer en général?
 - c) Quelle est la ou les tailles idéales de la liste de contrôle à utiliser lors d'une inspection?
 - d) Quelle est la ou les tailles idéales du code à inspecter?
 - e) Quels sont les effets d'un nombre plus élevé de personnes au sein d'une équipe d'inspection?
 - f) Quels employés avez-vous trouvé les plus efficaces à affecter à l'inspection et pourquoi?